

3. Manipulación Geométrica de la Imagen



Universidad de Valladolid

3. Manipulación Geométrica de la Imagen.

1. Introducción
2. Transformaciones Rígidas
 - Traslación
 - Rotación
 - Reflexión
3. Transformaciones Afines
 - Escalado
 - Cizalladura
 - Similitud
4. Transformaciones Proyectivas
 - Coordenadas Homogéneas
5. Combinación de Transformaciones
6. Interpolación
7. Transformaciones geométricas con MATLAB

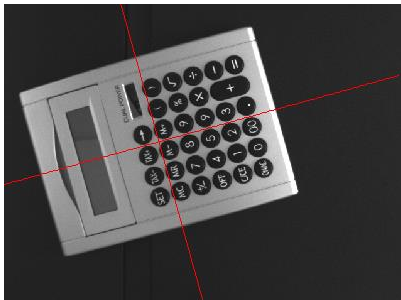
1. Introducción

- Empleando el histograma, obteníamos una transformación que asignaba para cada nivel de gris de la imagen de entrada un nuevo nivel de gris. Este tipo de transformaciones se llaman **puntuales** pues sólo hace falta conocer el nivel de gris en cada punto de la imagen de entrada para obtener el valor en el mismo punto de la imagen de salida.
- En este capítulo nos ocuparemos de las **transformaciones geométricas**. Determinaremos qué posición tomará en la imagen destino cada píxel de la imagen original cuando sobre ella aplicamos una transformación geométrica tales como traslación, rotación, escalado... Es decir, el valor de un píxel en la imagen de salida se asignará en base a las coordenadas (x,y) de ese píxel.

1. Introducción

Las transformaciones geométricas que veremos son no son distintas de las transformaciones básicas estudiadas en geometría. No obstante, debido a la naturaleza discreta de las imágenes, aparecen ciertos problemas que es preciso analizar y resolver.

Este tipo de transformaciones resultan útiles para facilitar el reconocimiento de formas cuando no existen unas condiciones preestablecidas de escala o posición en las piezas a analizar.



1. Introducción

- Las transformaciones geométricas también son utilizadas para eliminar distorsiones debidas a óptica y a la perspectiva o bien para reajustar imágenes de una misma escena tomadas bajo distintas condiciones y poder de esta forma establecer correspondencias entre unas y otras.



1. Introducción

- Podemos clasificar las transformaciones geométricas en:
- *Transformaciones rígidas o euclídeas*, que preservan las distancias, ángulos y áreas.
- *Transformaciones afines*, que preservan la colinealidad de los puntos, paralelismos y las razones entre los puntos pertenecientes a una línea.
- *Transformaciones proyectivas*, que preservan solo la colinealidad de los puntos.

1. Introducción

En transformaciones rígidas y afines las coordenadas de la imagen de salida (x', y') se obtienen a partir de la ecuación lineal en las coordenadas de la imagen original (x, y)

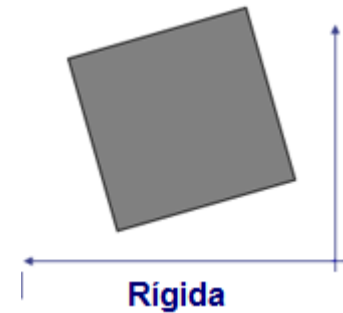
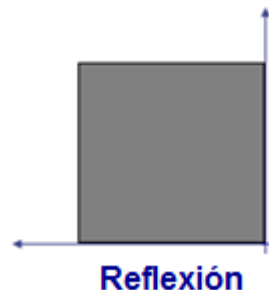
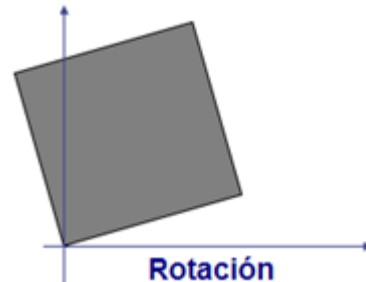
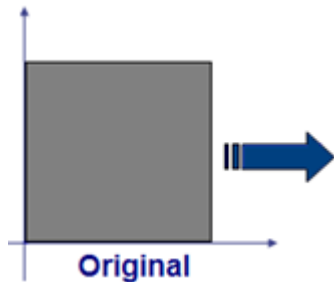
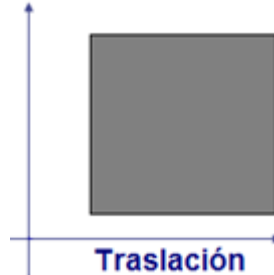
$$\begin{bmatrix} x' \\ y' \end{bmatrix} = \begin{bmatrix} m_{11} & m_{12} \\ m_{21} & m_{22} \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix} + \begin{bmatrix} t_x \\ t_y \end{bmatrix}$$
$$\mathbf{x}' = \mathbf{M}\mathbf{x} + \mathbf{t}$$

M debe cumplir la condición de ser invertible

2. Transformaciones Rígidas

Las transformaciones rígidas se caracterizan por preservar las distancias. M es una matriz ortogonal. Son transformaciones rígidas :

- Traslación
- Rotación
- Reflexión



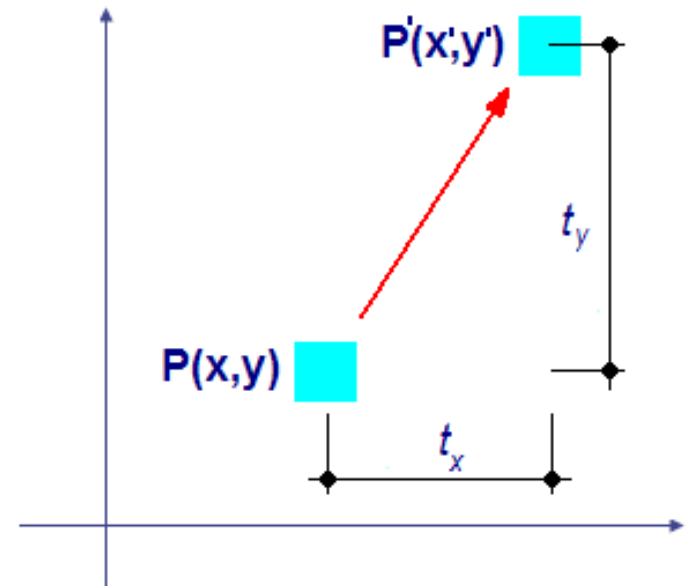
2. Transf. Rígidas. Traslación

La traslación es una transformación que desplaza una cierta magnitud vectorial cada uno de los píxeles de la imagen de entrada.

$$x' = x + t_x$$

$$y' = y + t_y$$

$$\begin{bmatrix} x' \\ y' \end{bmatrix} = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} \cdot \begin{bmatrix} x \\ y \end{bmatrix} + \begin{bmatrix} t_x \\ t_y \end{bmatrix}$$



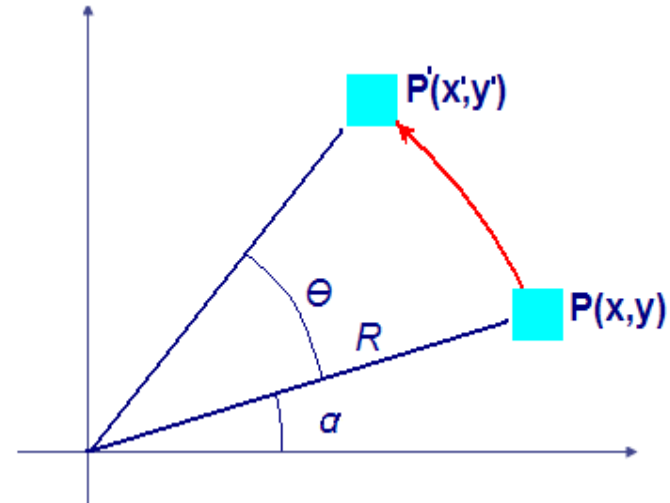
2. Transf. Rígidas. Rotación

La rotación consiste en girar la imagen original un cierto ángulo. La rotación en principio se establece respecto al origen de coordenadas

$$x' = x \cdot \cos(\theta) - y \cdot \sin(\theta)$$

$$y' = x \cdot \sin(\theta) + y \cdot \cos(\theta)$$

$$\begin{bmatrix} x' \\ y' \end{bmatrix} = \begin{bmatrix} \cos\theta & -\sin\theta \\ \sin\theta & \cos\theta \end{bmatrix} \cdot \begin{bmatrix} x \\ y \end{bmatrix}$$



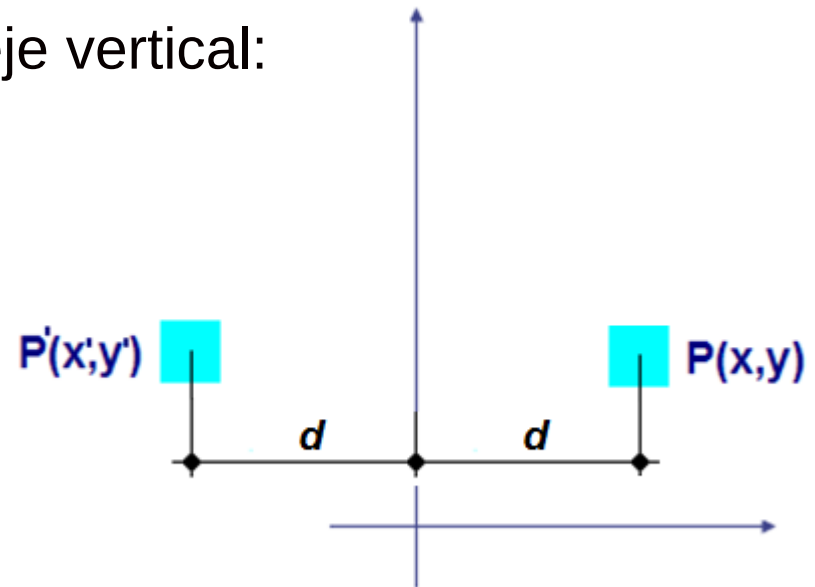
2. Transf. Rígidas. Reflexión

Dada una recta r y un punto P , la reflexión del punto $P = (x, y)$ respecto a la recta r genera un punto $P' = (x', y')$ caracterizado por:

- El vector $\overrightarrow{PP'}$ es perpendicular a la recta r
- Las distancias de P y P' a la recta son iguales

Ejemplo: reflexión respecto al eje vertical:

$$\begin{bmatrix} x' \\ y' \end{bmatrix} = \begin{bmatrix} -1 & 0 \\ 0 & 1 \end{bmatrix} \cdot \begin{bmatrix} x \\ y \end{bmatrix}$$



3. Transformaciones Afines

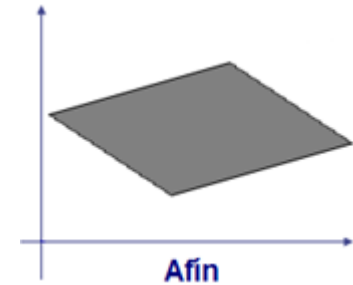
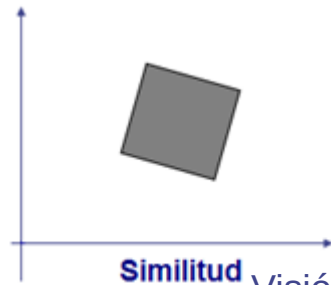
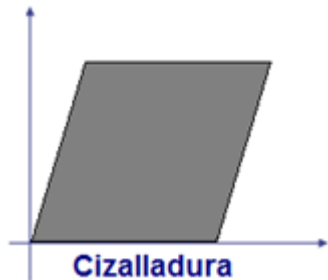
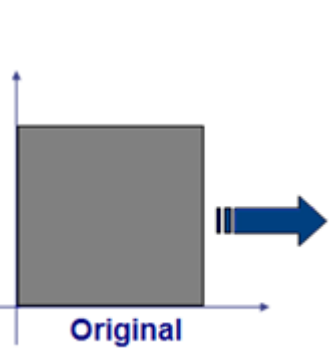
- Las transformaciones afines preservan la colinealidad de los puntos (las rectas siguen siendo rectas tras la transformación), el paralelismo y las razones entre los puntos de pertenecientes a una recta.

$$\begin{bmatrix} x' \\ y' \end{bmatrix} = \begin{bmatrix} m_{11} & m_{12} \\ m_{21} & m_{22} \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix} + \begin{bmatrix} t_x \\ t_y \end{bmatrix}$$
$$\mathbf{x}' = \mathbf{M}\mathbf{x} + \mathbf{t}$$

M es una matriz invertible.

3. Transformaciones Afines

- Las transformaciones afines incluyen:
 - Escalado
 - Cizalladura
 - Similitud



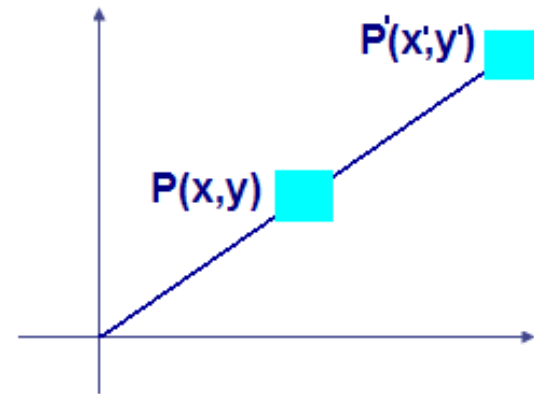
3. Transf. Afines. Escalado

El escalado es una transformación que se origina al multiplicar por un factor ambas coordenadas de cada píxel de la imagen de entrada.

$$x' = s_x \cdot x$$

$$y' = s_y \cdot y$$

$$\begin{bmatrix} x' \\ y' \end{bmatrix} = \begin{bmatrix} s_x & 0 \\ 0 & s_y \end{bmatrix} \cdot \begin{bmatrix} x \\ y \end{bmatrix}$$



El factor de escala no tiene necesariamente que ser el mismo para ambas coordenadas (escalado anisotrópico)

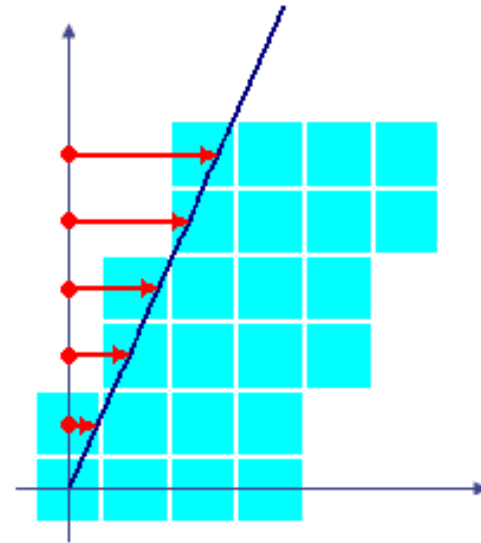
3. Transf. Afines. Cizalladura

La cizalladura de x respecto a y desplaza cada píxel de la imagen original en la dirección x un espacio proporcional a su coordenada y .

$$x' = x + c_x \cdot y$$

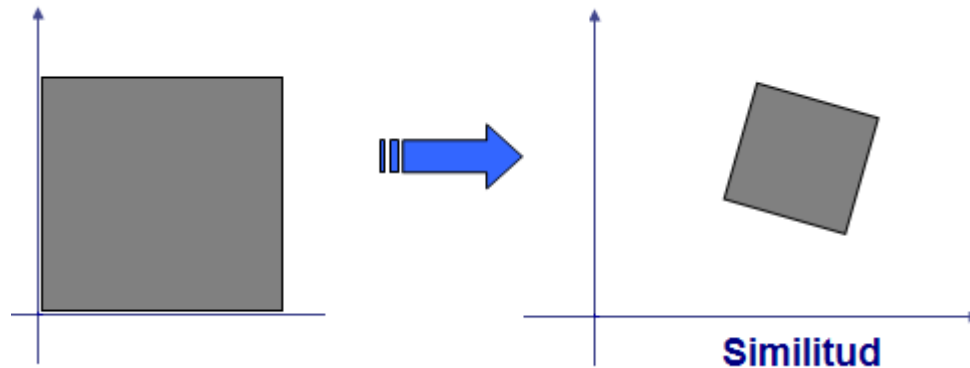
$$y' = y$$

$$\begin{bmatrix} x' \\ y' \end{bmatrix} = \begin{bmatrix} 1 & c_x \\ 0 & 1 \end{bmatrix} \cdot \begin{bmatrix} x \\ y \end{bmatrix}$$



3. Transf. Afines. Similitud

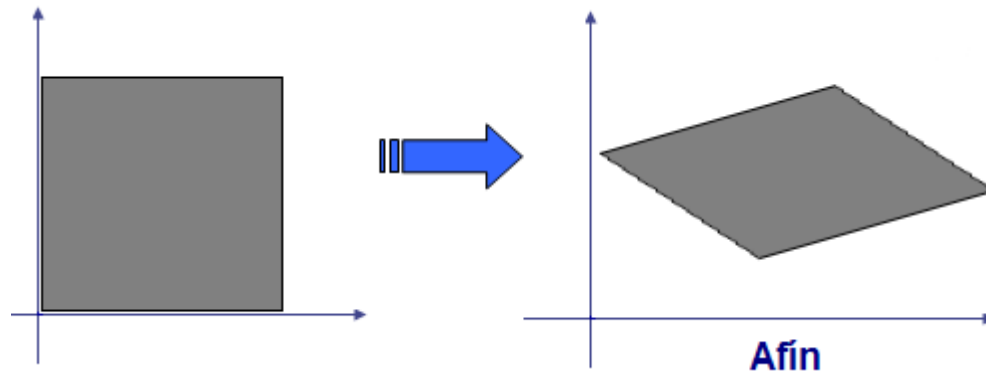
Similitud: Traslación + Rotación + Escalado Isotróp.



En las transformaciones afines de similitud se conservan también los ángulos

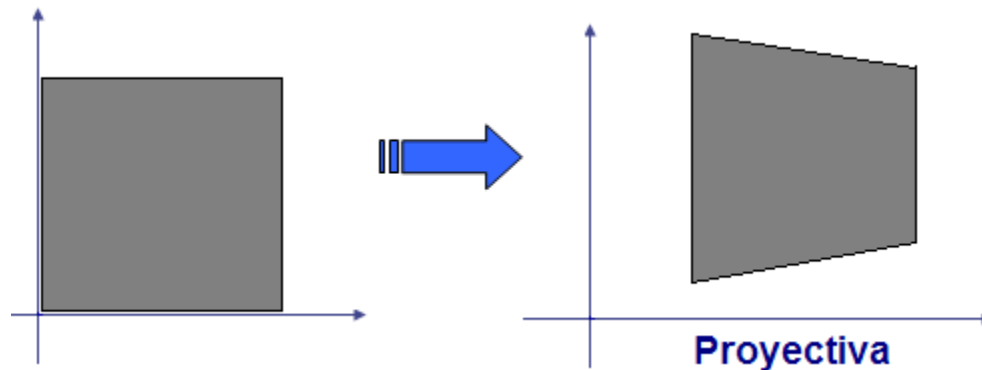
3. Transf. Afines. Caso general

Afín: Similitud + Escalado anisotrópico + Cizalladura

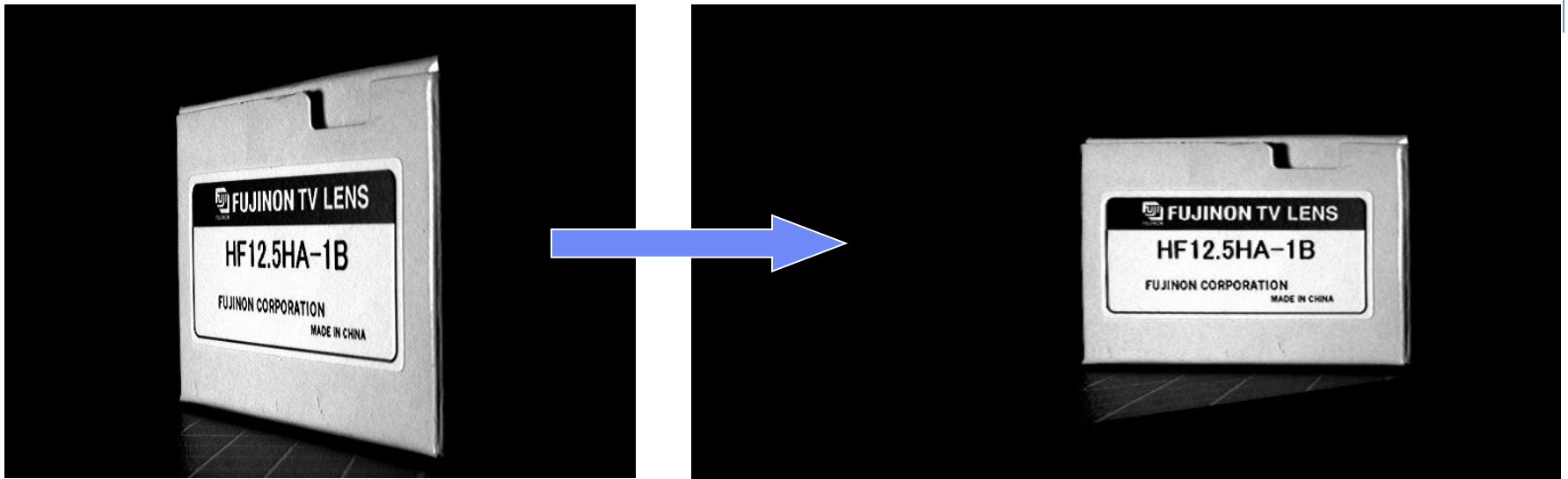


4. Transformaciones Proyectivas

En las transformaciones proyectivas ya no se conserva el paralelismo, ni las razones entre puntos de una recta. Sólo se conservan las líneas rectas.



4. Transformaciones Proyectivas



4. Transformaciones Proyectivas



4.1. Coordenadas Homogéneas

Si consideramos la expresión matricial de la traslación:

$$\begin{bmatrix} x' \\ y' \end{bmatrix} = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} \cdot \begin{bmatrix} x \\ y \end{bmatrix} + \begin{bmatrix} t_x \\ t_y \end{bmatrix}$$

Tiene una forma distinta del resto de las transformaciones pues no se reduce a un único producto de matrices sino que además contiene un sumando.

4.1. Coordenadas Homogéneas

- Interesa que todas las transformaciones tengan una representación uniforme mediante un producto de matrices. Esto permitirá operar más eficientemente, especialmente cuando hay que realizar una secuencia de transformaciones.
- Para lograr esta representación matricial uniforme recurriremos a la utilización de **coordenadas homogéneas**.
- En coordenadas homogéneas los puntos del plano se representan con tres coordenadas.

4.1. Coordenadas Homogéneas

Un punto (x, y) tiene la forma (hx, hy, h) , donde h toma un valor arbitrario distinto de 0 que representa un factor de escala.

Un mismo punto tiene infinitas representaciones en coordenadas homogéneas. El punto $(2, 3)$ puede expresarse como $(2, 3, 1)$, $(4, 6, 2)$, $(6, 9, 3)$, ...

No obstante, lo habitual es tomar $h=1$, con lo que el punto (x, y) pasa a ser $(x, y, 1)$

4.1. Coordenadas Homogéneas

La traslación se expresará entonces en coordenadas homogéneas de la forma:

$$\begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} 1 & 0 & t_x \\ 0 & 1 & t_y \\ 0 & 0 & 1 \end{bmatrix} \cdot \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

Y en general cualquier transformación afín como:

$$\begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} m_{11} & m_{12} & t_x \\ m_{21} & m_{22} & t_y \\ 0 & 0 & 1 \end{bmatrix} \cdot \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

4.1. Coordenadas Homogéneas

Traslación

$$\begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} 1 & 0 & t_x \\ 0 & 1 & t_y \\ 0 & 0 & 1 \end{bmatrix} \cdot \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

Rotación

$$\begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} \cos\theta & -\operatorname{sen}\theta & 0 \\ \operatorname{sen}\theta & \cos\theta & 0 \\ 0 & 0 & 1 \end{bmatrix} \cdot \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

Escalado

$$\begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} s_x & 0 & 0 \\ 0 & s_y & 0 \\ 0 & 0 & 1 \end{bmatrix} \cdot \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

Euclídea

$$\begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} \cos\theta & -\operatorname{sen}\theta & t_x \\ \operatorname{sen}\theta & \cos\theta & t_y \\ 0 & 0 & 1 \end{bmatrix} \cdot \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

Similitud

$$\begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} s \cdot \cos\theta & -s \cdot \operatorname{sen}\theta & t_x \\ s \cdot \operatorname{sen}\theta & s \cdot \cos\theta & t_y \\ 0 & 0 & 1 \end{bmatrix} \cdot \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

5. Combinación de transformaciones

Cuando se aplican dos o más transformaciones de forma consecutiva, estas se pueden combinar en una única transformación sin más que multiplicar las matrices de transformación. Esta es otra de las grandes ventajas de trabajar con **coordenadas homogéneas**.

Ejemplo dos traslaciones:

$$T_2 T_1 = \begin{bmatrix} 0 & 0 & t_{2x} \\ 0 & 0 & t_{2y} \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 0 & 0 & t_{1x} \\ 0 & 0 & t_{1y} \\ 0 & 0 & 1 \end{bmatrix} = \begin{bmatrix} 0 & 0 & t_{1x} + t_{2x} \\ 0 & 0 & t_{1y} + t_{2y} \\ 0 & 0 & 1 \end{bmatrix} = T_{12}$$

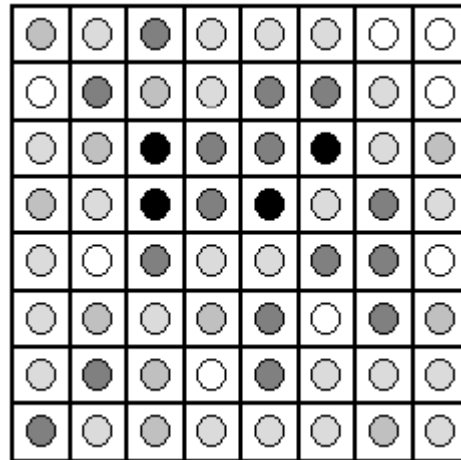
5. Combinación de transformaciones

En el caso anterior, el orden en que se efectúen las traslaciones no tiene importancia pero en general sí que hay que tener en cuenta el **orden** en que se hacen las operaciones.

En general, el producto de las matrices de transformación no será conmutativo. Las **matrices de transformaciones posteriores irán multiplicando por la izquierda a las transformaciones previas.**

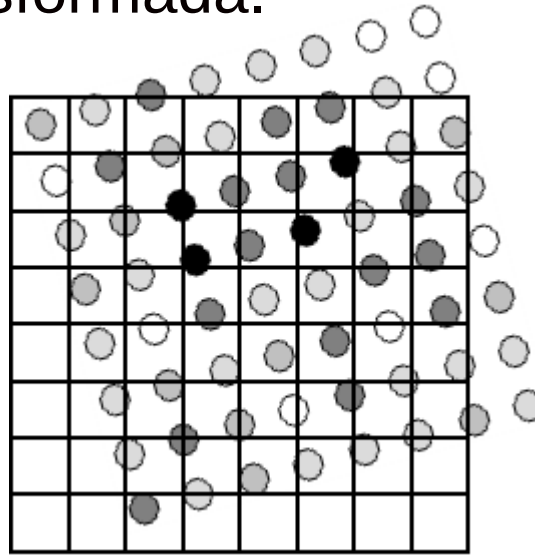
6. Interpolación

Cuando trabajamos sobre **imágenes digitales**, estas **no son continuas** sino que están integradas por píxeles cuya posición se representa por unas coordenadas que siempre deben tomar valores discretos.



6. Interpolación

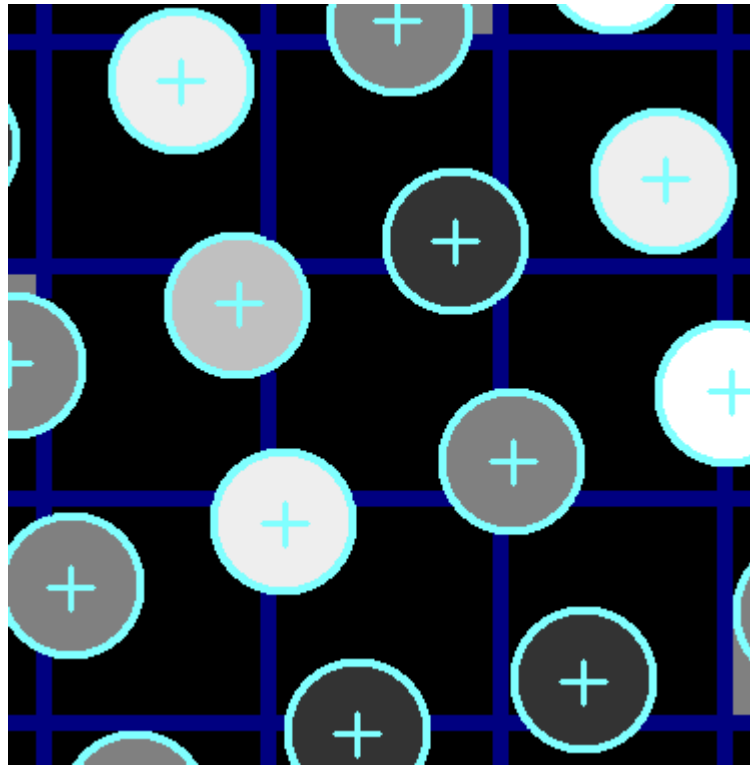
La transformación geométrica de una imagen originará unas nuevas coordenadas para los niveles de gris que en general **no coincidirán exactamente sobre la retícula** de la imagen transformada.



Las nuevas coordenadas tomarán valores reales, no enteros como precisan los píxeles en una imagen.

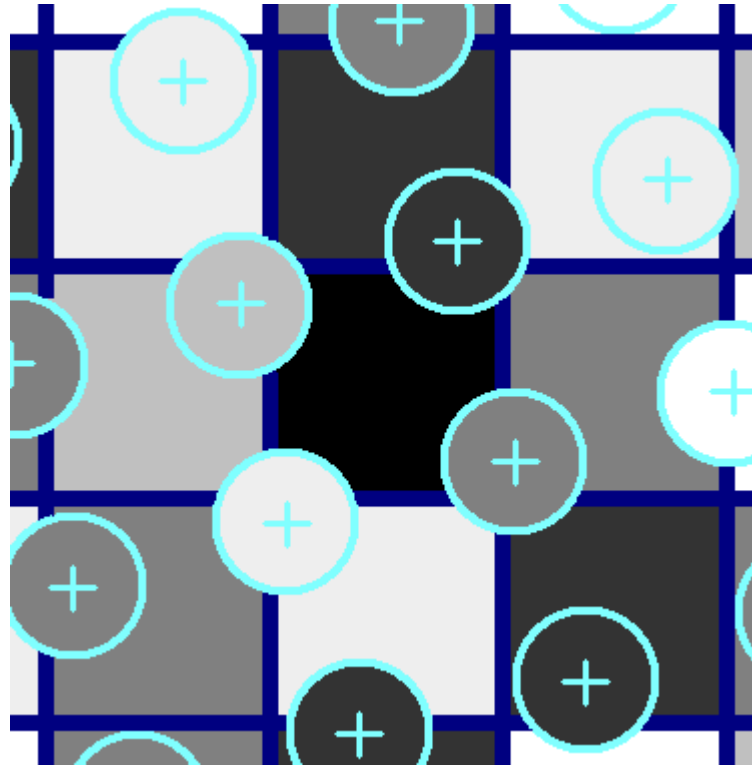
6. Interpolación

Podríamos optar por redondear estas coordenadas a la **posición del pixel más próximo**



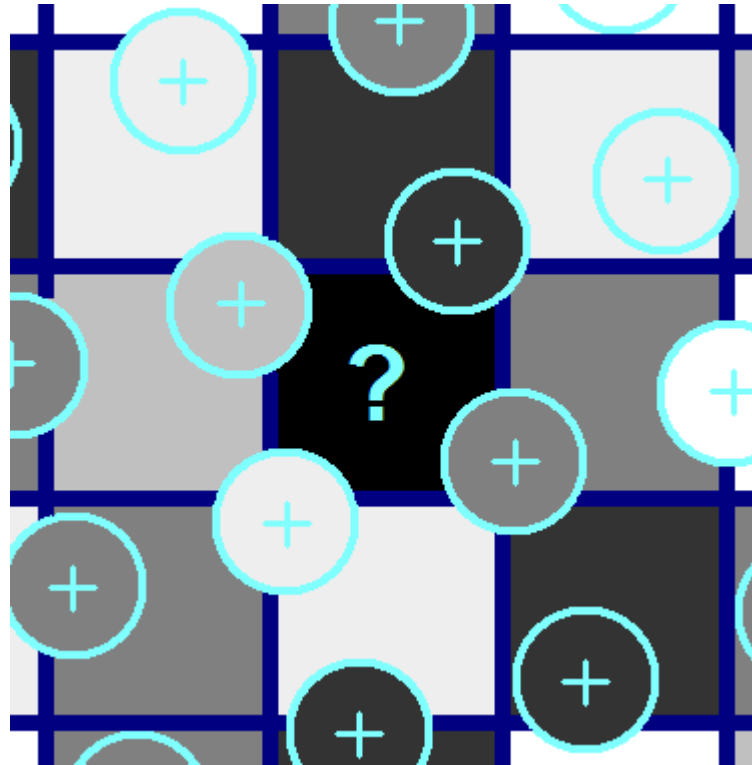
6. Interpolación

Podríamos optar por redondear estas coordenadas a la **posición del pixel más próximo**



6. Interpolación

Pero ¿qué pasa con aquellas posiciones que no son próximas a ningún píxel en la imagen transformada?



6. Interpolación

Ejemplo: una rotación de 15° sobre la imagen siguiente empleando las expresiones presentadas y redondeando las coordenadas.



Imagen original

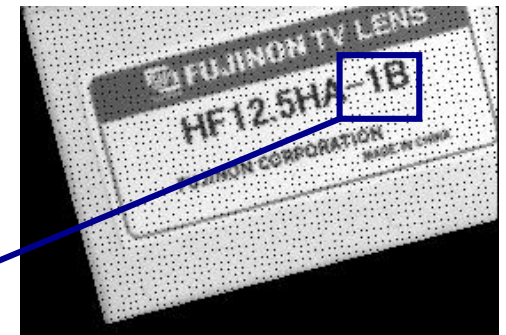
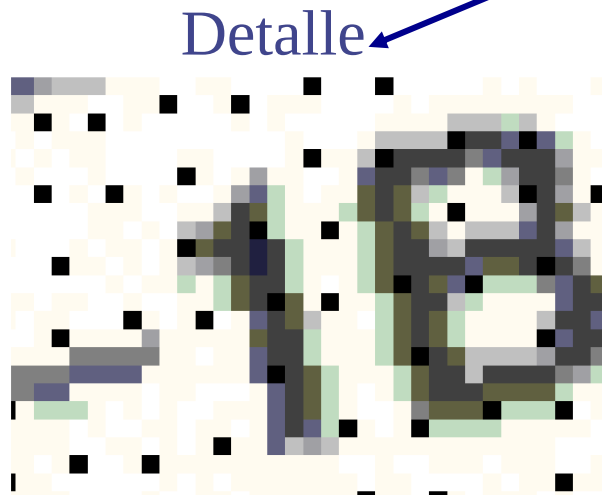


Imagen resultante de rotar 15 grados



Detalle

6. Interpolación.

Hay píxeles a los que no se les asigna ningún nivel de gris porque todas las coordenadas transformadas en su entorno están más próximas a los píxeles adyacentes que a él .

La **transformación directa** que hemos aplicado tiene varios inconvenientes:

- Puede haber píxeles a los que no se les asigne ningún valor.
- Puede haber píxeles a los que se les asigna sucesivamente varios niveles de gris.
- Se transforman píxeles que luego caerán fuera de la imagen cuyo cálculo es innecesario.

6.1. Transformación inversa.

Es más conveniente a partir de las dimensiones de la imagen de salida, recorrerla haciendo la **transformación inversa** para ver qué píxel de la imagen de entrada sería el que corresponde en la imagen original.

Así se ahorra tiempo al no tener que calcular píxeles fuera de la imagen y, sobre todo, se evita el efecto de los indeseables puntos negros ya que ahora todos los píxeles de la imagen de salida son considerados y se les asignará un valor obtenido a partir de la imagen de entrada.

Para la transformación inversa emplearemos la matriz inversa de la transformación

6.1. Transformación inversa.

Cuando se aplicamos la transformación inversa las coordenadas resultantes tomarán generalmente valores con decimales.

Podemos determinar el nivel de gris que correspondería a cualquier par de coordenadas reales a partir del nivel de gris de los píxeles adyacentes por medio de una interpolación.

Básicamente, la calidad de la interpolación dependerá del número de píxeles adyacentes que se consideren. Cuanto más píxeles se consideren en el entorno de la interpolación mejor va a ser esta. Sin embargo esta calidad será a expensas de un coste computacional más elevado.

6.2. Interpolación. Algoritmos

Existen muchos algoritmos de interpolación. Consideraremos los más comunes en tratamiento de imágenes industrial donde se requiere una carga computacional que no sea excesiva para una ejecución en tiempo real:

- Interpolación por el vecino más próximo.
- Interpolación bilineal.
- Interpolación bicúbica.

¡Ojo! La interpolación no es una operación reversible: sucesivas interpolaciones, por buenas que sean, hacen que se degrade la calidad de la imagen.

6.2. Interpolación al vecino más próximo

La forma más sencilla de interpolar es como se ha hecho anteriormente, redondeando las coordenadas obtenidas en la transformación para asignar el nivel de gris del píxel con esas coordenadas enteras. Esta interpolación se conoce como “*vecino más próximo*”.

Con la interpolación por el vecino más próximo se soluciona rápidamente el problema de la interpolación pero con un resultado bastante pobre porque las imágenes resultantes aparecerán muy pixeladas y con contornos muy escalonados (*aliasing*).

6.2. Interpolación al vecino más próximo



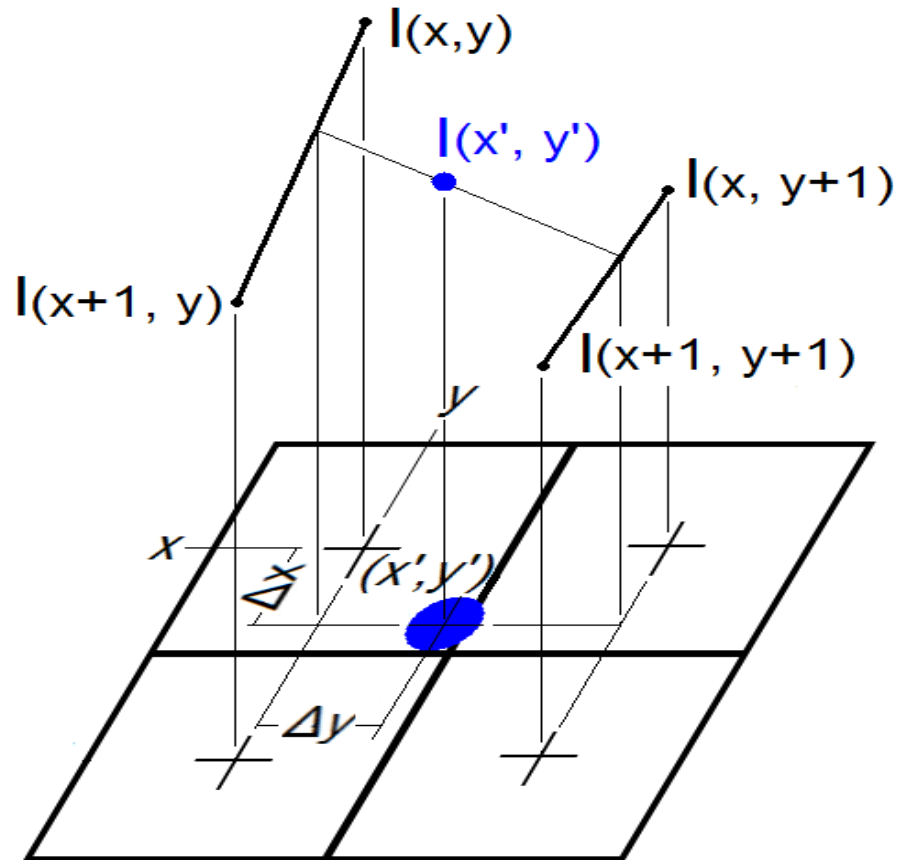
6.2. Interpolación bilineal

La **interpolación bilineal** hace una estimación del nivel de gris del píxel basándose en la vecindad 2x2 que rodea al píxel.

El nivel de gris se obtiene a partir de la media ponderada de los píxeles de la vecindad. Los píxeles de esta vecindad más cercanos a la posición a estimar tendrán un mayor peso que los que estén más lejos.

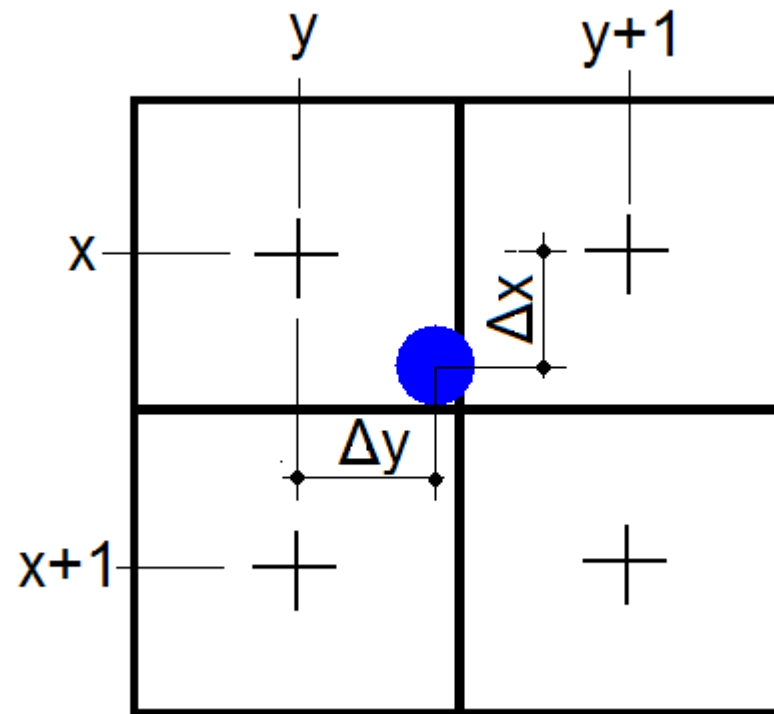
El resultado es una imagen con una apariencia más suave que las obtenidas con el vecino más próximo.

6.2. Interpolación bilineal



La **interpolación bilineal** consiste en dos interpolaciones lineales sucesivas, una en cada variable .

6.2. Interpolación bilineal



$$I(x', y') = \frac{(1 - \Delta x)(1 - \Delta y)I(x, y) + \Delta x(1 - \Delta y)I(x + 1, y) + \Delta y(1 - \Delta x)I(x, y + 1) + \Delta x \Delta y I(x + 1, y + 1)}{(1 - \Delta x)(1 - \Delta y) + \Delta x(1 - \Delta y) + \Delta y(1 - \Delta x) + \Delta x \Delta y}$$

6.2. Interpolación bicúbica

La **interpolación bicúbica** emplea la misma filosofía que la bilineal pero va un poco más allá al considerar una vecindad 4×4 en torno a la posición a estimar. De los 16 píxeles considerados, los niveles de gris de los más próximos tendrán una ponderación más alta que los más lejanos.

La interpolación bicúbica produce imágenes de mayor calidad que los dos métodos anteriores. La técnica resulta una buena opción cuando se requiera de buena calidad en la imagen interpolada pero presenta un mayor coste computacional.

6.2. Comparativa Algoritmos Interpolación



Vecino más
próximo



Bilineal



Original



Bicúbica



7. Transf. geométricas con MATLAB

```
>> R = imrotate(I, angGrados,'bilinear');
```

Rota la imagen I el ángulo especificado en grados con interp.bilineal.

```
>> T = maketform('affine',t);
```

Crea una estructura de datos para aplicar la transformación geométrica.

```
>> J = imtransform(I,T);
```

Aplica la transformación geométrica a la imagen I especificada en la estructura T.

```
>> C = imcrop(I,[x0 y0 ancho alto]);
```

Recorta de la imagen I la ventana especificada y la guarda en C.

```
>> E = imresize(I,2,'bilinear');
```

Reescala la imagen I con un factor 2 usando una interpolación bilineal.